Decentralized Federated Learning Marketplace

End-to-End System Architecture, Real-World Use Cases, and Audit Workflow

Project Scope: Privacy-Preserving Collaborative AI with Ethereum Smart Contracts & MongoDB Audit Trail
Document Version: 1.0 (Production Blueprint) | **Author:** Federated Learning Research & Dev Team

---

1. Executive Summary & The Problem It Solves

Modern Artificial Intelligence thrives on massive datasets. However, in sensitive industries—such as healthcare, banking, defense, and edge computing—centralizing data in a single cloud database is practically impossible and often illegal due to strict data privacy regulations (GDPR, HIPAA, DPDP) and corporate secrecy. This creates isolated **data silos**, preventing organizations from training robust models.

The Solution: This project introduces a **Decentralized Federated AI Marketplace**. Rather than bringing raw data to the central model, our architecture **brings the model to the data**. Participants train locally on private data, share only mathematical parameter updates (weights), record cryptographic proofs (SHA-256 hashes) on the Ethereum blockchain for zero-trust auditability, receive automated token rewards via smart contracts, and log rich telemetry into an off-chain MongoDB database.

The Core Paradigm Shift:
• *Traditional Centralized AI:* Raw Private Data → Sent across Internet → Central Server (High privacy & breach risk).
• *Federated AI Marketplace:* Raw Data stays on local device → Local Training → Send mathematical weights & blockchain hashes → Collaborative global intelligence without ever exposing raw samples.

2. Intuition Through Real-World Examples

To understand how every part of the system operates, consider these three industry scenarios:

Example A: Healthcare & Multi-Hospital Cancer Diagnosis

Imagine three independent oncology centers: **Hospital A** (specialized in rare lung tumors), **Hospital B** (large urban pediatric center), and **Hospital C** (regional research clinic). Due to patient confidentiality (HIPAA), none can legally send patient MRI or CT scans to each other or to an external cloud server.

Workflow in our Marketplace:

1. The central coordinator publishes an initial global tumor detection model.
2. Hospital A, B, and C download this model and train it solely on their local patient scans.
3. Each hospital computes its local model update and calculates a SHA-256 fingerprint of the weights.
4. The fingerprint is submitted to our **Ethereum Smart Contract** to guarantee timestamped proof of contribution.
5. The full parameters and training stats (accuracy, loss, scan count) are stored in **MongoDB**.
6. The smart contract validates the participation and automatically distributes incentive tokens.
7. The central server aggregates updates using **FedAvg** to create a smarter global model capable of detecting rare tumors that no single hospital had enough data to identify on its own.

Example B: Financial Fraud Detection Across Competing Banks

Credit card fraudsters intentionally spread fraudulent transactions across multiple banks (e.g., Chase, Citi, and Wells Fargo) to avoid triggering single-bank thresholds. Banks cannot legally or competitively share their clients' transaction books.

With this federated platform, each bank trains locally on its proprietary fraud logs. The collective model learns coordinated cross-bank attack vectors without any bank revealing customer identities, balances, or purchasing history.

Example C: Mobile Keyboards & Edge Devices (Smartphone Privacy)

When typing on a smartphone, next-word prediction algorithms learn your vocabulary, slang, and typing habits. If all keystrokes were transmitted to cloud servers, passwords and private messages would be exposed. Instead, your phone trains the prediction model locally while charging overnight, uploading only weight adjustments.

3. System Architecture: The Four Core Pillars

The project is structured into four interconnected modules, each solving a distinct engineering challenge:

Pillar / Layer	Technology	Primary Responsibility	Codebase Location
1. Machine Learning Engine	Python, NumPy, PyTorch, Scikit-Learn	Simulates decentralized edge nodes, Non-IID data distribution, local gradient descent, SHA-256 fingerprinting, and Federated Averaging (FedAvg).	federated-ai-marketplace/ml-engine/
2. Blockchain Consensus Layer	Solidity, Hardhat, Ganache, Ethers.js	Zero-trust registry: enforces participant eligibility, records immutable hashes of model updates, and dispenses programmatic incentive tokens/rewards.	federated-ai-marketplace/blockchain/
3. Off-Chain Database Layer	MongoDB, Mongoose ORM	High-throughput persistence for client profiles, training metrics (accuracy, loss curves, dataset size), timestamped round history, and verification caching.	database/ backend/models/
4. Orchestration API Backend	Node.js, Express, REST API	Coordinates data flow between ML nodes, MongoDB, and Blockchain. Performs cryptographic cross-audits to detect tampered or withheld updates.	federated-ai-marketplace/backend/

4. Why Both Blockchain and MongoDB? (The Hybrid Storage Pattern)

A common architectural question in Web3 AI systems is: *Why not store everything on the blockchain? Or why not store everything in MongoDB?* Our design follows modern decentralized engineering best practices by splitting state:

Criteria	Why Blockchain Alone Fails	Why MongoDB Alone Fails	Our Hybrid Synergy
Storage Cost & Limits	Storing a 50MB deep learning model on Ethereum costs millions in gas and exceeds block gas limits.	Free/cheap storage, but centralized.	Only 32-byte **SHA-256 hashes** go to Blockchain; full weights & metrics go to MongoDB.
Immutability & Trust	Immutable, tamper-proof, decentralized.	A malicious database admin can alter training metrics or fake participant records.	Blockchain acts as an untamperable **notary**; MongoDB must match on-chain hashes.
Query Performance	Blockchain cannot efficiently query historical ranges, sort by accuracy, or join collections.	Sub-millisecond queries, secondary indexes, aggregation pipelines.	Rich UI and analytics query MongoDB; audits cross-reference on-chain logs.
Incentives & Rules	Smart contracts execute autonomous token transfers without middlemen.	Databases cannot natively transfer cryptographic currency or tokens.	Smart contract rewards valid workers upon verification.

5. Step-by-Step System Workflow & Execution Cycle

Each round of federated learning follows a strict six-phase lifecycle coordinated across the components:

Phase 1: Participant Onboarding & KYC Registration

When a client node (e.g., client_0, client_1) joins the marketplace, it generates an Ethereum wallet address. The client sends a registration request to the backend. The backend records the client profile in MongoDB (isRegisteredOnChain: false) and invokes the smart contract's registerParticipant(clientAddress) function. Once confirmed by Ethereum miners/Ganache, the resulting transaction hash is saved in MongoDB, marking the client as officially certified.

Phase 2: Local Non-IID Training at the Edge

The client downloads the current global model weights. Using its private local partition of data (which is intentionally skewed/Non-IID to reflect realistic uneven distributions), the node runs local forward and backward propagation (epochs of mini-batch gradient descent) to minimize local loss.

Phase 3: Cryptographic Fingerprinting (Model Hashing)

Before sending any parameter adjustments over the network, the client serializes its updated weight matrices into a canonical JSON representation and computes a deterministic **SHA-256 hash**. This hash represents the immutable digital fingerprint of that exact training computation.

Phase 4: Dual-Write Submission (Off-Chain + On-Chain)

The client sends its update payload (metrics, loss, accuracy, dataset size, and model hash) to POST /api/update. The backend executes a atomic dual-write:

- **Off-Chain (MongoDB):** Appends the update record to the active round document in the rounds collection.
- **On-Chain (Blockchain):** Calls submitUpdateHash(round, clientAddress, modelHash) on the smart contract.

Phase 5: Automated Validation & Token Incentive Payout

Once the hash is anchored to the blockchain, the coordinator evaluates the quality of the contribution (accuracy improvement and gradient integrity). If valid, the backend invokes validateAndReward(round, clientAddress, true) on the smart contract, minting/transferring financial reward tokens directly to the client's wallet.

Phase 6: Federated Averaging (FedAvg) & Global Aggregation

Once all participating clients complete the round, the central aggregator applies the standard Federated Averaging formula:

$$W_{global} = \Sigma (n_k / n) * W_k$$

where n_k is the dataset size of client k, and n is the total samples across all nodes. The global model is updated, hashed, recorded in MongoDB, and broadcasted as the starting baseline for Round N+1.

6. Cryptographic Cross-Audit Mechanism (/api/verify/:roundId)

The cornerstone of trust in our marketplace is the independent cross-audit endpoint. Anyone (a client, regulator, or auditor) can query GET /api/verify/:roundId to prove that the off-chain database has not been tampered with.

The Verification Algorithm:
1. Fetch all update records for Round N from the **MongoDB** rounds collection.
2. Fetch all on-chain events from the **Ethereum Smart Contract** via getUpdatesForRound(N).
3. For each client, compare mongoUpdate.modelHash === blockchainUpdate.modelHash.
4. If all hashes match: **Status = VERIFIED (Tamper-Proof)**.
5. If any hash differs or is missing on-chain: **Status = AUDIT FAILED (Data Tampered)**.

7. Codebase Directory & File-by-File Blueprint

Here is the precise mapping of source code files in the repository and their exact operational role:

File Path	Role & Key Responsibilities
ml-engine/main.py	Orchestrates multi-client federated training simulation across 5 rounds. Manages local epoch loops, calls registration, submits updates, hashes weights, and calls FedAvg.
ml-engine/model.py	Defines the neural network architecture (NumPy/PyTorch MLP) with forward passes, backprop, parameter serialization, and weight extraction.
ml-engine/dataset.py	Generates synthetic classification datasets and splits them into distinct Non-IID partitions for client nodes to simulate real-world data heterogeneity.
blockchain/contracts/ FederatedLearningRegistry.sol	Solidity smart contract with mappings for participants, roundUpdates, and rewards. Emits events for real-time auditability.
blockchain/scripts/deploy.js	Hardhat deployment script that compiles the Solidity contract and deploys it to the local Ganache network, outputting the active contract address.
backend/server.js	Main Express HTTP server. Connects to MongoDB via Mongoose. Exposes /api/register, /api/update, /api/validate, /api/round/:id, and /api/verify/:id.
backend/services/blockchain.js	Ethers.js bridge that signs Ethereum transactions. Includes resilient fallbacks to ensure training proceeds even during network interruptions.
backend/models/Client.js	Mongoose schema for client participants: tracks clientId, address, registration status, timestamps, and on-chain tx hashes.
backend/models/Round.js	Mongoose schema for training rounds: tracks roundId, globalModelHash, and nested updates array (client, modelHash, datasetSize, accuracy, loss, txHash).

8. Project Status & Upcoming UI Integration

Current Status:

- Core ML simulation (Non-IID, FedAvg, gradient descent) is fully functional.
- Smart contract registry and reward distribution tested on Ganache testnet.
- MongoDB persistence and cross-verification audit endpoints fully operational.
- Resilient fallback architecture prevents network disruptions from halting runs.

Next Deliverables (Frontend Dashboard):

The upcoming UI will connect directly to the REST API and Ethereum provider to display:

1. **Live Training Monitor:** Real-time accuracy and loss convergence charts across rounds.
2. **Node Marketplace Map:** Active participating nodes, their stake, and reputation scores.
3. **Blockchain Audit Explorer:** Interactive transaction verification table highlighting matched vs mismatched SHA-256 hashes.
4. **Wallet & Reward Hub:** Visual balance of distributed participant tokens.

Specification Approved for Academic & Prototype Review
This document serves as the comprehensive architectural and procedural reference for the Federated AI Marketplace project.